Zombi Escape

Diagrama de clases UML · Proyecto programado II · Programación Orientada a Objetos · TEC

Contenido

Diagrama	Paquetes	Qué muestra
1. Modelo y lógica del juego	`modelo`, `logica`	Jugadores (herencia Jugador → Superviviente / Sniper), zombies, mapa, la Partida con sus reglas y un hilo por zombie.
2. Red: cliente y servidor	`red.Servidor`, `red.Cliente`, `red.Models`	Servidor con un hilo por cliente, protocolo de mensajes y la "foto" EstadoJuego que viaja por el socket.
3. Interfaz gráfica	`gui`, `red.Cliente`	Login, ventana de juego, panel del mapa y el hilo que envía los movimientos.

Las clases que ya aparecen completas en otro diagrama se muestran solo con su nombre y la nota «ver diagrama N».

Notación

Símbolo	Significado
`+` `-` `#` `~`	Visibilidad: público, privado, protegido, de paquete
subrayado	Atributo o método estático (constantes)
cursiva	Método abstracto (debe implementarlo la clase hija)
Flecha con triángulo, línea continua	Herencia (`extends`)
Flecha con triángulo, línea punteada	Implementa una interfaz (`implements`)
Rombo negro	Composición: la parte no existe sin el todo (Partida crea y destruye sus zombies e hilos)
Rombo blanco	Agregación: el todo usa partes que existen por su cuenta (Partida y sus jugadores)
Flecha continua	Asociación: la clase guarda una referencia a la otra
Flecha punteada	Dependencia: la usa o la crea, sin guardarla
«interface», «abstract», «enumeration»	Tipo especial de clase
«Java»	Clase de la biblioteca de Java (Thread, JFrame, JPanel)

Decisiones de diseño principales

Herencia y polimorfismo	`Jugador` es abstracta; `getImagen()` y `puedeMoverse()` los define cada hija, así la ventana y la partida no necesitan `instanceof`.
Interfaz IMovible	La comparten `Superviviente` y `Zombie`, que se mueven por el mapa aunque no son el mismo tipo de objeto.
Observador	`Partida` avisa los cambios por `EscuchaPartida` (la implementa el servidor) y el cliente avisa a la ventana por `EscuchaCliente`. La lógica no depende de la red ni de la interfaz.
Hilos	Un `HiloZombie` por zombie; un `ThreadServidor` por cliente; `HiloMovimiento` envía los pasos a ritmo fijo sin congelar la ventana.
Servidor autoritativo	El cliente solo pide (MOVE, SHOOT, AIM, HIDE); el servidor valida con `Partida` y envía a todos un `EstadoJuego`.

Generado a partir del código fuente compilado (javap), por lo que los atributos y métodos coinciden con el código entregado. Se omiten los componentes visuales generados por NetBeans y las clases sin uso (MainCliente, ZombieEscape, PruebaMapa).

Modelo y lógica del juego

Partida

+String ESPERANDO
+String JUGANDO
+String NIVEL_SUPERADO
+String DERROTA
+String VICTORIA
-String[] SKINS
-int PUNTOS_PRIMERO
-int PUNTOS_MENOS
-int PUNTOS_MINIMO
-int PUNTOS_ZOMBIE
-int PUNTOS_RESCATE
-int FALLO_MIN
-int FALLO_MAX
-int VELOCIDAD_MIN
-int VELOCIDAD_MAX
-int MARGEN_ENTRADA
-int PAUSA_ENTRE_NIVELES
-ArrayList<Mapa> niveles
-EscuchaPartida escucha
-Random azar
-Mapa mapa
-int nivelActual
-String estado
-ArrayList<Jugador> jugadores
-ArrayList<Zombie> zombies
-ArrayList<HiloZombie> hilos
-long inicioNivel
-int segundosFinales
-int llegadas

+Partida(ArrayList<Mapa>, EscuchaPartida)
+agregarJugador(Jugador) : void
+quitarJugador(Jugador) : void
+iniciarNivel(int) : void
-colocar(Jugador, int) : void
-crearZombies() : void
-lejosDeLaEntrada(Posicion, ArrayList<Posicion>) : boolean
+moverJugador(Jugador, Direccion) : void
+esconder(Jugador) : void
+apuntar(Jugador, int, int) : void
+disparar(Jugador, int, int) : String
+moverZombie(Zombie) : void
-revisarRescates(Superviviente) : void
-revisarTarjetas() : void
-revisarDetecciones() : void
-revisarFinNivel() : void
-programarSiguienteNivel() : void
-continuarConNivel(int) : void
-detenerZombies() : void
+terminar() : void
-zombieEn(Posicion) : Zombie
-contarSupervivientes() : int
-calcularSegundos() : int
-notificar() : void
+getMapa() : Mapa
+getEstado() : String
+getNivel() : int
+getTotalNiveles() : int
+getJugadores() : ArrayList<Jugador>
+getZombies() : ArrayList<Zombie>
+getSegundos() : int
+getRanking() : ArrayList<Jugador>

«Java»
Thread

«interface»
IMovible

+mover(Mapa) : void
+getDireccion() : Direccion

CargadorMapas

+String CARPETA

+CargadorMapas()
+cargar(String) : Mapa
+cargarTodos(String) : ArrayList<Mapa>

«interface»
EscuchaPartida

+partidaCambio() : void

HiloZombie

-Zombie zombie
-Partida partida
-boolean activo

+HiloZombie(Zombie, Partida)
+run() : void
+detener() : void

«abstract»
Jugador

+int ESPERA_MOVIMIENTO
-int TOLERANCIA
#String nombre
#Posicion posicion
#int puntos
#boolean vivo
#long ultimoMovimiento

+Jugador(String)
+getNombre() : String
+getPosicion() : Posicion
+getPuntos() : int
+isVivo() : boolean
+setPosicion(Posicion) : void
+sumarPuntos(int) : void
+morir() : void
+revivir() : void
+reiniciarPuntaje() : void
+listoParaMover() : boolean
+getImagen() : String
+puedeMoverse() : boolean

Mapa

-String nombre
-int deteccion
-int cantidadZombies
-char[][] mapa

+Mapa(String, int, int, char[][])
+getNombre() : String
+getDeteccion() : int
+getCantidadZombies() : int
+getFilas() : int
+getColumnas() : int
+estaDentro(int, int) : boolean
+getCelda(int, int) : char
+esCaminable(int, int) : boolean
+esEscondite(int, int) : boolean
+esSalida(int, int) : boolean
+buscarCeldas(char) : ArrayList<Posicion>
+esCaminable(Posicion) : boolean
+esEscondite(Posicion) : boolean
+esSalida(Posicion) : boolean
+esCaminableZombie(Posicion) : boolean

Zombie

-Random azar
-int PROBABILIDAD_DOBLAR
-Posicion posicion
-Direccion direccion
-boolean vivo
-int velocidad

+Zombie(Posicion, Direccion, int)
+getPosicion() : Posicion
+getDireccion() : Direccion
+isVivo() : boolean
+getVelocidad() : int
+morir() : void
+mover(Mapa) : void
+detecta(Superviviente, Mapa) : boolean
+getImagen() : String

Superviviente

+int DURACION_TARJETA
+int PROTECCION
-Direccion direccion
-Direccion movimientoPendiente
-boolean escondido
-boolean escapo
-String skin
-long momentoMuerte
-boolean tarjetaActiva
-long protegidoHasta
-int segundosEscape
-int tiempoTotal

+Superviviente(String)
+getDireccion() : Direccion
+isEscondido() : boolean
+isEscapo() : boolean
+setSkin(String) : void
+isTarjetaActiva() : boolean
+isProtegido() : boolean
+morir() : void
+revisarVencimientoTarjeta() : void
+revivir() : void
+pedirMovimiento(Direccion) : void
+mover(Mapa) : void
+alternarEscondite(Mapa) : void
+reiniciar(Posicion) : void
+registrarEscape(int) : void
+getSegundosEscape() : int
+getTiempoTotal() : int
+reiniciarPuntaje() : void
+getImagen() : String
+puedeMoverse() : boolean

Sniper

+int BALAS_POR_ESCENARIO
+int RADIOVISION
-int countBalas

+Sniper(String)
+getBalas() : int
+tieneBalas() : boolean
+gastarBala() : void
+recargar() : void
+apuntarA(Posicion, Mapa) : void
+puedeVer(Posicion) : boolean
+getImagen() : String
+puedeMoverse() : boolean

TipoCelda

+char PARED
+char CAMINO
+char ESCONDITE
+char OBSTACULO
+char ENTRADA
+char SALIDA

+TipoCelda()

Posicion

-int fila
-int columna

+Posicion(int, int)
+getFila() : int
+getColumna() : int
+siguiente(Direccion) : Posicion
+esIgual(Posicion) : boolean
+toString() : String
+distancia(Posicion) : int

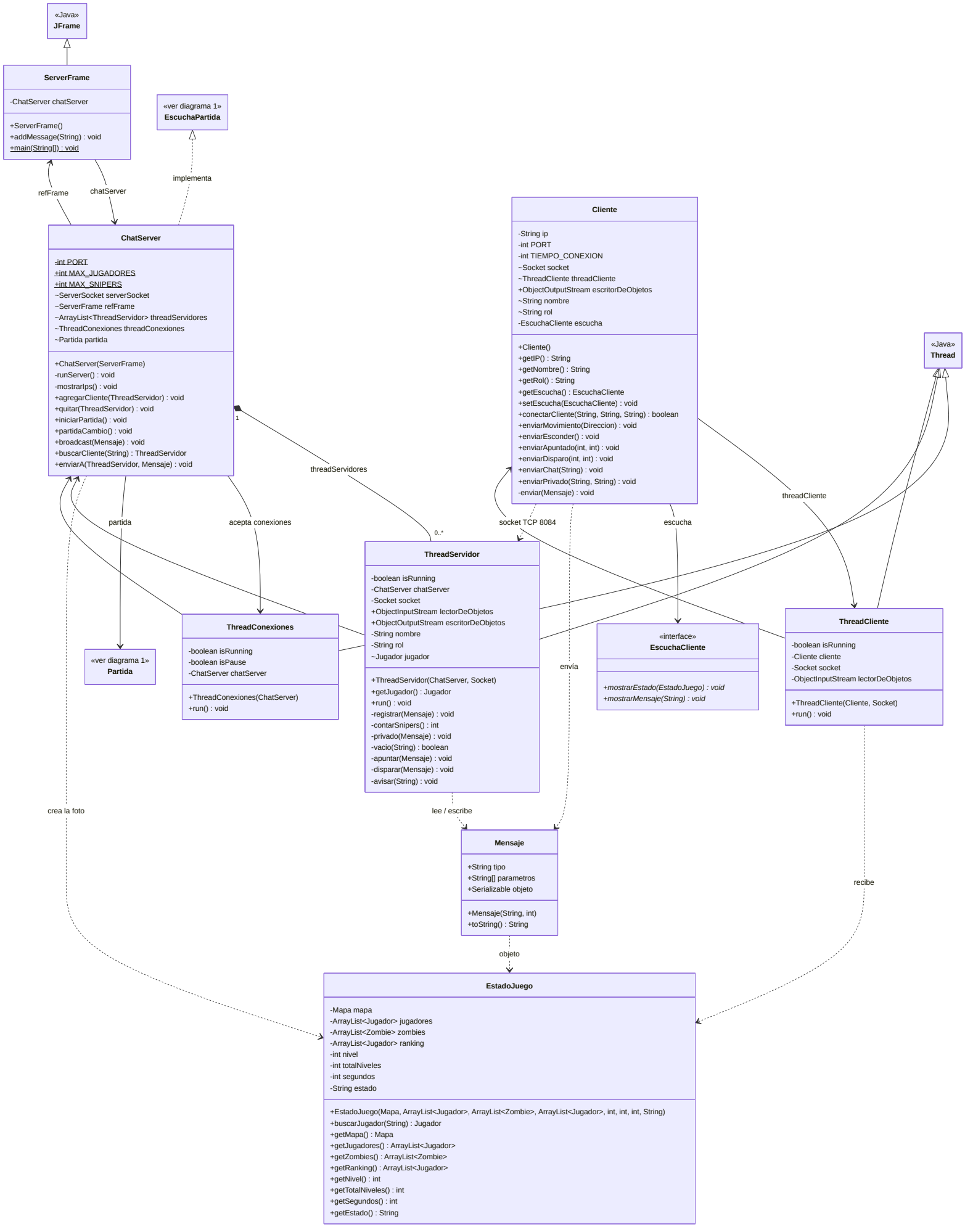
«enumeration»
Direccion

ARRIBA
ABAJO
IZQUIERDA
DERECHA
-int dx
-int dy

-Direccion(int, int)
+getDx() : int
+getDy() : int
+girarDerecha() : Direccion
+girarIzquierda() : Direccion
+opuesta() : Direccion
+textoAMovimiento(String) : Direccion
+getNombre() : String

1
1
niveles
jugadores
0..10
1
1
avisa cambios
hilos
partida
0..*
zombies
crea
mueve
implementa
implem
1..
hereda
hereda
posicion
símbolos
buscarCeldas()
posicion
direccion
direccion
siguiente()

Red: cliente y servidor (sockets)



Interfaz gráfica (cliente)

«Java»
JFrame

VentanaLogin

-Cliente cliente
+String nombre
+String rol
-int MAX_NOMBRE
-Color VERDE
-Color VERDE_CLARO

+VentanaLogin()
-iniciarJuego(String, String, String) : void
+main(String[]) : void

«ver diagrama 2»
EscuchaCliente

implementa

abre

VentanaJuego

-Cliente cliente
-String miNombre
-String miRol
-HiloMovimiento hiloMovimiento
-boolean espacioPresionado
-int nivelMostrado
-boolean apuntando
-String TODOS
-int MAX_CHAT
-ArrayList<String> destinos
-String estadoAnterior
-Color VERDE
-Color ROJO
-Color AMARILLO
-PanelMapa panelMapa

+VentanaJuego(Cliente, String, String)
-esSniper() : boolean
-configurarTeclas() : void
-configurarEspacio(InputMap, ActionMap) : void
-enlazarFlecha(InputMap, ActionMap, int, Direccion) : void
-escribiendo() : boolean
-configurarChat() : void
-enviarChat() : void
-actualizarDestinos(EstadoJuego) : void
-configurarMouseSniper() : void
-dejarDeApuntar() : void
+mostrarEstado(EstadoJuego) : void
+mostrarMensaje(String) : void
-actualizar(EstadoJuego) : void
-actualizarAviso(EstadoJuego) : void
-miSuperviviente(EstadoJuego) : Superviviente
-mostrarRanking(EstadoJuego) : void
-actualizarSniper(EstadoJuego) : void
-crearFiguras(EstadoJuego) : ArrayList<Figura>
-textoJugadores(EstadoJuego) : String
-situacion(Jugador) : String
-textoEstado(EstadoJuego) : String
-formatoTiempo(int) : String
-escaparHtml(String) : String

«Java»
Thread

«Java»
JPanel

cliente

hiloMovimiento

panelMapa

crea

HiloMovimiento

-Cliente cliente
-boolean activo
-ArrayList<Direccion> presionadas
-Direccion toque

+HiloMovimiento(Cliente)
+presionar(Direccion) : void
+soltar(Direccion) : void
+soltarTodo() : void
-siguienteDireccion() : Direccion
+detener() : void
+run() : void

PanelMapa

+int TAM
-String CARPETA_IMG
-Mapa mapa
-ArrayList<Figura> figuras
-Posicion centroVision
-int radioVision
-Posicion apuntada
-Color NIEBLA
-Color ROJO_MIRA
-String avisoTitulo
-String avisoDetalle
-Color avisoColor
-boolean avisoCentrado
-HashMap<String, Image> imagenes

+PanelMapa()
+setMapa(Mapa) : void
+setFiguras(ArrayList<Figura>) : void
+setVision(Posicion, int) : void
+setAviso(String, String, Color, boolean) : void
+quitarAviso() : void
+setApuntada(Posicion) : void
+celdaEn(int, int) : Posicion
-dentroDeVision(int, int) : boolean
-getImagen(String) : Image
-nombreImagenCelda(char) : String
#paintComponent(Graphics) : void
-dibujarAviso(Graphics2D) : void
-dibujarNiebla(Graphics2D) : void
-dibujarEtiqueta(Graphics2D, String, int, int) : void

cliente

envía MOVE

1
figuras
0..*

«ver diagrama 2»
Cliente

Figura

-String imagen
-int fila
-int columna
-String etiqueta
-boolean oculta

+Figura(String, int, int, String, boolean)
+getImagen() : String
+getFila() : int
+getColumna() : int
+getEtiqueta() : String
+isOculta() : boolean